Diagnosing and Resolving SQL Server PAGEIOLATCH Wait Types

Detection · Root-Cause Analysis · Remediation · Verification

Document Type	Technical SOP / Runbook
Platform	SQL Server 2016 – 2025 (Box, EC2, Azure VM)
Domain	Performance Tuning · Storage I/O
Classification	Internal — Operational Use
Version	1.0
Source	https://www.mssqltips.com/sqlservertip/11611/understanding-sql-server-pageiolatch-wait-types/
Credits To	M A A Mehedi Hasan

Document Control

Field	Detail
Title	SOP – Diagnosing and Resolving SQL Server PAGEIOLATCH Wait Types
Owner	Database Engineering (SQL DBA Champs)
Author	Praveen Madupu, Sr. SQL Server / Cloud DBA
Applies To	On-prem, AWS EC2, and Azure VM SQL Server instances
Review Cycle	Semi-annual, or after any storage / memory topology change
Status	Approved for operational use

Revision History

Version	Date	Summary of Change
1.0	2026-07-12	Initial publication of the PAGEIOLATCH diagnostic and remediation SOP.

Contents

1. Purpose and Scope

This Standard Operating Procedure defines a repeatable, evidence-based method for detecting, diagnosing, and resolving PAGEIOLATCH wait activity on SQL Server. PAGEIOLATCH waits are one of the most common and most misdiagnosed symptoms a DBA encounters: they surface as "the database is slow," yet the true cause may lie in the query layer, the memory configuration, or the storage subsystem. Treating the wait type as the disease rather than the symptom leads to wasted spend on hardware that never fixes the problem.

The objective of this SOP is to give the on-call DBA a decision tree that separates the three root-cause families — inefficient query plans, memory pressure, and slow storage — and prescribes the correct remediation for each, followed by a mandatory verification step so that no change is declared successful without measured proof.

In Scope

- Standalone and Always On Availability Group instances running SQL Server 2016 through 2025.
- Instances hosted on physical hardware, AWS EC2, and Azure VM (IaaS).
- Diagnosis via wait statistics, query-store / plan-cache DMVs, memory DMVs, and OS-level disk latency counters.

Out of Scope

- Platform-as-a-Service offerings (Azure SQL Database, Azure SQL Managed Instance, AWS RDS) where OS-level PerfMon access and DBCC buffer commands are restricted. The query-tuning sections still apply; the storage and OS sections do not.
- General wait-type triage for non-I/O waits (CXPACKET, LCK_M_*, SOS_SCHEDULER_YIELD, and similar), which are covered by their own SOPs.

2. Audience and Prerequisites

This procedure is written for DBAs and platform engineers who hold at least sysadmin or the granular VIEW SERVER STATE / ALTER SERVER STATE permissions on the target instance. Readers are expected to be comfortable with T-SQL, execution plans, and the Windows Performance Monitor.

Tooling

Tool	Purpose	Notes
SSMS 19+ / Azure Data Studio	Run diagnostic queries and inspect plans	Graphical plan viewer required
SqlQueryStress	Reproduce I/O load in a controlled test	Erik Ejlskov Jensen fork; lab / non-prod only
Windows Performance Monitor	Capture physical disk latency	Requires OS access (IaaS / box only)
dbatools (PowerShell)	Automate collection at scale across the estate	Optional but recommended

CAUTION — Production safety
Several diagnostic commands in this SOP clear server-wide caches (DBCC DROPCLEANBUFFERS, DBCC FREEPROCCACHE) or reset cumulative counters (DBCC SQLPERF ... CLEAR). These are destructive to the warm cache and to your baseline. Run them only in a lab, or during an approved maintenance window with change control. Never paste them into a production window "just to test."

3. Background: What a PAGEIOLATCH Wait Actually Is

SQL Server stores data in 8 KB pages and keeps recently accessed pages in memory in the buffer pool. When a query needs a page that is not resident in the buffer pool, SQL Server must issue a physical read to fetch it from disk. To keep that page structurally consistent while the I/O is in flight, the engine places a lightweight, non-configurable synchronization object — a latch — on the buffer frame. The worker thread that requested the page then waits until the read completes and the latch is granted.

That wait is recorded as a **PAGEIOLATCH** wait. The critical distinction to internalize is this: a latch protects a data structure in memory, whereas a lock protects logical data (rows, tables) for transactional consistency. A **PAGEIOLATCH** wait specifically means "a thread is stalled waiting for a data or index page to arrive from storage." It is fundamentally a signal that reads are hitting disk instead of memory.

> **NOTE — Latch vs. buffer latch**
> PAGELATCH_* (no "IO") waits protect in-memory pages that are already resident — typically tempdb allocation contention. PAGEIOLATCH_* (with "IO") waits are always tied to a physical I/O operation. Confusing the two sends you troubleshooting the wrong subsystem, so always read the wait name carefully.

3.1 The PAGEIOLATCH Sub-Types

The engine reports several PAGEIOLATCH sub-types depending on the latch mode requested while the page is being read. For diagnostic purposes you will overwhelmingly see the shared and exclusive variants; the others are useful context.

Sub-type	Latch mode	When it appears
PAGEIOLATCH_SH	Shared	A read operation is pulling a page from disk so it can be read. Most common on read-heavy / reporting workloads.
PAGEIOLATCH_EX	Exclusive	A page is being read from disk in order to be modified. Common on write-heavy / OLTP workloads.
PAGEIOLATCH_UP	Update	Held while a page is read prior to being written back to disk.
PAGEIOLATCH_KP	Keep	Signals that the page must be kept in the buffer pool and protected from being evicted while referenced.
PAGEIOLATCH_DT	Destroy	Held before a page is destroyed / removed from the buffer pool.

> **NOTE — What matters for triage**
> The sub-type tells you the workload shape (SH = reads dominate, EX = writes dominate) but not the root cause. High PAGEIOLATCH of any sub-type points to the same three-way investigation: query, memory, or disk. Do not over-index on the specific letter pair.

4. Diagnostic Workflow at a Glance

Follow the phases in order. Each phase either confirms or eliminates a root-cause family, so that by the end you know which remediation section to apply. Do not jump straight to "buy faster disks" — that is the last hypothesis, not the first.

Phase	Question answered	Primary evidence
A – Confirm	Is PAGEIOLATCH actually a top wait?	sys.dm_os_wait_stats ranked by wait_time

Phase	Question answered	Primary evidence
B – Query	Are inefficient plans forcing excess reads?	Plan cache: missing indexes, scans, stale stats
C – Memory	Is the buffer pool starved?	Memory DMVs, Page Life Expectancy
D – Storage	Is the disk itself slow?	Avg. Disk sec/Read & /Write, virtual file stats
E – Verify	Did the fix reduce the wait?	Re-measured wait_time delta + latency

5. Phase A — Confirm PAGEIOLATCH Is a Top Wait

Before investigating anything, establish that PAGEIOLATCH is genuinely material relative to total signal. A few milliseconds of accumulated wait on an instance that has been up for months is noise. Rank all waits by accumulated time and confirm PAGEIOLATCH sits near the top.

5.1 Rank All Wait Types

The following query summarizes cumulative waits since the last service restart (or the last counter clear), filtering out benign background waits so the meaningful ones rise to the top.

```
/*  Top waits by accumulated wait time, excluding benign background waits.
    Read cumulatively -- values are since instance start / last DBCC SQLPERF CLEAR. */
SELECT TOP (20)
    ws.wait_type,
    ws.waiting_tasks_count                                  AS task_count,
    ws.wait_time_ms                                         AS total_wait_ms,
    ws.wait_time_ms - ws.signal_wait_time_ms               AS resource_wait_ms,
    ws.signal_wait_time_ms                                  AS cpu_signal_ms,
    ws.max_wait_time_ms,
    CAST(100.0 * ws.wait_time_ms
         / SUM(ws.wait_time_ms) OVER ()  AS DECIMAL(5,2))    AS pct_of_total
FROM sys.dm_os_wait_stats AS ws
WHERE ws.wait_type NOT IN (
        N'CLR_SEMAPHORE', N'LAZYWRITER_SLEEP', N'RESOURCE_QUEUE',
        N'SLEEP_TASK', N'SLEEP_SYSTEMTASK', N'SQLTRACE_BUFFER_FLUSH',
        N'WAITFOR', N'BROKER_TASK_STOP', N'BROKER_TO_FLUSH',
        N'CHECKPOINT_QUEUE', N'REQUEST_FOR_DEADLOCK_SEARCH',
        N'XE_TIMER_EVENT', N'XE_DISPATCHER_WAIT', N'DIRTY_PAGE_POLL',
        N'HADR_FILESTREAM_IOMGR_IOCOMPLETION', N'DISPATCHER_QUEUE_SEMAPHORE',
        N'SP_SERVER_DIAGNOSTICS_SLEEP', N'FT_IFTS_SCHEDULER_IDLE_WAIT')
  AND ws.waiting_tasks_count > 0
ORDER BY ws.wait_time_ms DESC;
```

5.2 Isolate the PAGEIOLATCH Family

Once you have confirmed it is prominent, drill into just the PAGEIOLATCH sub-types to see the read/write split and the average wait per occurrence — the average is more actionable than the raw total.

```
SELECT
    ws.wait_type,
    ws.waiting_tasks_count                                      AS task_count,
    ws.wait_time_ms                                             AS total_wait_ms,
    CAST(ws.wait_time_ms * 1.0
         / NULLIF(ws.waiting_tasks_count, 0) AS DECIMAL(10,2))      AS avg_wait_ms,
    ws.max_wait_time_ms
FROM sys.dm_os_wait_stats AS ws
WHERE ws.wait_type LIKE N'PAGEIOLATCH[_]%'
```

```
ORDER BY ws.wait_time_ms DESC;
```

NOTE — Interpreting the average

An average wait per task above roughly 15–20 ms is a strong hint the storage path is slow or overloaded. A low average wait combined with a very high task count usually points the other way — the workload is simply reading far more pages than it should, which is a query or memory problem, not a disk problem.

6. Phase B — Rule Out Inefficient Queries

In the majority of real incidents, PAGEIOLATCH waits are a downstream effect of the query layer reading far more pages than necessary. A plan that scans a large table instead of seeking a targeted range forces the engine to pull thousands of pages from disk, driving physical reads and, in turn, PAGEIOLATCH waits. Fixing the plan removes the I/O entirely — which is always cheaper than making the I/O faster. Investigate this family first.

6.1 Common Query-Layer Culprits

- **Missing indexes.** A large scan where a narrow seek would suffice.
- **Stale statistics.** The optimizer mis-estimates cardinality and chooses a scan-heavy plan.
- **Implicit conversions.** A data-type mismatch in a predicate silently disables an otherwise usable index.
- **Non-SARGable predicates.** Wrapping a column in a function (e.g. filtering on a computed expression) prevents index seeks.
- **Implicit / unnecessary reads.** SELECT * and wide row-by-row patterns that touch pages the query never needed.

6.2 Find the High-I/O Plans With Missing Indexes

This query surfaces the most CPU- and I/O-expensive cached plans and flags those whose execution plan contains a missing-index recommendation. Treat these as the primary tuning backlog.

```
WITH XMLNAMESPACES (N'http://schemas.microsoft.com/sqlserver/2004/07/showplan' AS p)
SELECT TOP (25)
    DB_NAME(st.dbid)                                          AS database_name,
    qs.execution_count,
    (qs.total_logical_reads + qs.total_logical_writes)        AS total_logical_io,
    (qs.total_logical_reads + qs.total_logical_writes)
        / NULLIF(qs.execution_count, 0)                       AS avg_logical_io,
    qs.total_physical_reads,
    qs.total_worker_time / 1000                               AS total_cpu_ms,
    qs.last_execution_time,
    SUBSTRING(st.text,
        (qs.statement_start_offset / 2) + 1,
        ((CASE qs.statement_end_offset WHEN -1
              THEN DATALENGTH(st.text)
              ELSE qs.statement_end_offset END
          - qs.statement_start_offset) / 2) + 1)              AS query_text,
    qp.query_plan
FROM sys.dm_exec_query_stats AS qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle)      AS st
CROSS APPLY sys.dm_exec_query_plan(qs.plan_handle)   AS qp
WHERE qp.query_plan.exist(N'//p:MissingIndexGroup') = 1
ORDER BY (qs.total_logical_reads + qs.total_logical_writes) DESC;
```

BEST PRACTICE — Validate before you create

Missing-index DMV suggestions are hints, not commands. Each candidate must be evaluated against the existing index set, write cost, and overlap with other suggested indexes. Consolidate overlapping recommendations, keep key columns narrow, and never blindly create every suggestion — over-indexing shifts the pain to writes and page splits.

6.3 Detect Stale Statistics

Outdated statistics are a frequent, low-visibility cause of scan-heavy plans. The query below lists user-table statistics ordered by age and by the number of modifications accumulated since the last update, so you can target refreshes where they matter.

```
SELECT
    OBJECT_SCHEMA_NAME(s.object_id)                         AS schema_name,
    OBJECT_NAME(s.object_id)                                AS table_name,
    s.name                                                  AS stat_name,
    STATS_DATE(s.object_id, s.stats_id)                     AS last_updated,
    DATEDIFF(DAY, STATS_DATE(s.object_id, s.stats_id),
             GETDATE())                                     AS days_old,
    sp.rows,
    sp.modification_counter                                 AS mods_since_update,
    s.auto_created,
    s.user_created,
    s.no_recompute
FROM sys.stats AS s
CROSS APPLY sys.dm_db_stats_properties(s.object_id, s.stats_id) AS sp
WHERE OBJECTPROPERTY(s.object_id, 'IsUserTable') = 1
  AND (s.auto_created = 1 OR s.user_created = 1)
ORDER BY sp.modification_counter DESC, days_old DESC;
```

BEST PRACTICE — Remediation guidance

Refresh targeted statistics with UPDATE STATISTICS ... WITH FULLSCAN on the specific object where cardinality estimates are wrong, rather than a blanket sp_updatestats across the database. Consider enabling the automatic-tuning / stats maintenance job in your regular maintenance window, and ensure AUTO_UPDATE_STATISTICS remains ON.

6.4 Confirm the Fix in the Plan

After adding an index or refreshing statistics, re-run the offending query with SET STATISTICS IO ON and confirm that logical and physical reads drop and that the plan now shows a seek rather than a scan. A lower page count directly translates into fewer PAGEIOLATCH waits.

7. Phase C — Rule Out Memory Pressure

If the queries are already efficient yet PAGEIOLATCH persists, the next hypothesis is that the buffer pool is too small to hold the working set. When there is not enough memory to cache hot pages, SQL Server must repeatedly evict and re-read them from disk — physical reads that would never occur on a correctly sized instance. This manifests as PAGEIOLATCH even when every plan is optimal.

7.1 Compare Committed vs. Target Memory

The engine tracks how much memory it currently holds (committed) versus how much it wants (target). The relationship between the two is a fast read on whether the instance is under memory pressure.

```
SELECT
    sm.total_physical_memory_kb      / 1024   AS total_physical_mb,
    sm.available_physical_memory_kb  / 1024   AS available_physical_mb,
    si.committed_kb                  / 1024   AS sql_committed_mb,
```

```
    si.committed_target_kb          / 1024   AS sql_target_mb,
    sm.system_memory_state_desc
FROM sys.dm_os_sys_memory  AS sm
CROSS JOIN sys.dm_os_sys_info AS si;
```

Observation	Interpretation	Action
Committed ≈ Target	Memory is stable; SQL Server has what it asked for.	Memory is not the bottleneck — move to Phase D.
Committed < Target	SQL Server still has room to grow into.	Watch as workload grows; likely not the current cause.
Committed > Target	OS-level memory pressure; SQL Server is being asked to release memory.	Investigate external memory consumers; review Max Server Memory.

7.2 Corroborate With Buffer-Pool Health

Do not rely on a single counter. Page Life Expectancy (PLE) measures how long, in seconds, a page stays in the buffer pool before eviction; a low or sharply falling PLE alongside high PAGEIOLATCH is a classic memory-pressure fingerprint. The buffer-cache hit ratio and total pages give supporting context.

```
SELECT
    (SELECT cntr_value FROM sys.dm_os_performance_counters
       WHERE counter_name = N'Page life expectancy'
         AND object_name LIKE N'%Buffer Manager%')           AS ple_seconds,
    (SELECT cntr_value FROM sys.dm_os_performance_counters
       WHERE counter_name = N'Database pages'
         AND object_name LIKE N'%Buffer Manager%')           AS database_pages,
    (SELECT cntr_value FROM sys.dm_os_performance_counters
       WHERE counter_name = N'Total Server Memory (KB)') / 1024 AS total_server_mem_mb,
    (SELECT cntr_value FROM sys.dm_os_performance_counters
       WHERE counter_name = N'Target Server Memory (KB)') / 1024 AS target_server_mem_mb;
```

CAUTION — PLE thresholds are not universal

The old “PLE should be above 300” rule is obsolete on modern large-memory servers. Judge PLE against your own baseline and against NUMA-node buffer pools individually. A large drop from your normal steady-state is more meaningful than any absolute number.

7.3 Remediation Options

- Right-size Max Server Memory so SQL Server can cache the working set while leaving headroom for the OS and other services.
- Add physical / VM memory where the working set genuinely exceeds installed RAM.
- Identify and relocate competing memory consumers on the same host (other instances, application services, antivirus).
- Reduce the working set at the source by fixing the Phase B query inefficiencies — fewer pages read means fewer pages to cache.

8. Phase D — Rule Out a Slow Disk Subsystem

Only after queries and memory have been cleared should storage latency be treated as the root cause. When the queries are efficient and the buffer pool is adequately sized yet PAGEIOLATCH persists with a high average wait per task, the storage path

itself is too slow. Backups, index rebuilds, and other bulk operations can also cause transient spikes — distinguish a sustained problem from a scheduled one.

8.1 Measure Disk Latency From Inside SQL Server

The virtual file stats DMV gives per-file read and write latency without needing OS access, which makes it the most portable disk-latency measure and the one to start with.

```
SELECT
    DB_NAME(vfs.database_id)                                    AS database_name,
    mf.name                                                     AS logical_name,
    mf.type_desc,
    vfs.num_of_reads,
    vfs.num_of_writes,
    CAST(vfs.io_stall_read_ms  * 1.0
         / NULLIF(vfs.num_of_reads, 0)  AS DECIMAL(10,1))       AS avg_read_latency_ms,
    CAST(vfs.io_stall_write_ms * 1.0
         / NULLIF(vfs.num_of_writes, 0) AS DECIMAL(10,1))       AS avg_write_latency_ms,
    vfs.io_stall_read_ms,
    vfs.io_stall_write_ms,
    mf.physical_name
FROM sys.dm_io_virtual_file_stats(NULL, NULL) AS vfs
JOIN sys.master_files AS mf
     ON vfs.database_id = mf.database_id
    AND vfs.file_id     = mf.file_id
ORDER BY avg_read_latency_ms DESC;
```

8.2 Corroborate With PerfMon (IaaS / Box Only)

On instances where you have OS access, confirm the finding with the physical-disk latency counters in Windows Performance Monitor. These measure the storage path end-to-end, independent of SQL Server.

- `Avg. Disk sec/Read` — average read latency per I/O.
- `Avg. Disk sec/Write` — average write latency per I/O.

Latency (per I/O)	Assessment	Typical action
< 5 ms	Healthy for flash / SSD-backed storage.	Storage is not the bottleneck.
5 – 10 ms	Acceptable but worth watching.	Baseline and monitor for drift.
10 – 20 ms	Elevated — latency is contributing to waits.	Investigate queue depth, throttling, contention.
> 20 ms	Slow — storage is a primary contributor.	Engage storage/cloud team; review tier and layout.

NOTE — Cloud caveat

On AWS EC2 and Azure VM, sustained latency and throughput are governed by the disk SKU and the VM size. An undersized EBS volume type / gp3 IOPS setting, or an Azure disk tier below the workload's needs, or hitting the VM's uncached-disk throughput cap, will all present as slow storage from SQL Server's point of view. Check the provisioned IOPS/throughput and the VM limits before assuming the disk itself is faulty.

8.3 Storage Layout Best Practices

BEST PRACTICE — Storage configuration checklist

Use flash / SSD-class storage for all SQL Server data, log, and tempdb volumes.

Separate data files and transaction-log files onto different volumes (ideally different physical paths) to isolate sequential log writes from random data I/O.

Place tempdb on its own high-performance volume; on cloud VMs, a local NVMe / ephemeral disk is often ideal for tempdb.

For write-heavy, performance-critical workloads on-prem, prefer RAID 10 over parity RAID for its superior write profile.

Format data volumes with a 64 KB NTFS allocation unit size to align with SQL Server extents.

On cloud IaaS, right-size the disk tier (provisioned IOPS / throughput) to the measured workload and verify the VM series can sustain that throughput.

9. Phase E — Verify the Remediation

No change is complete until it has been measured. The verification step protects against the common failure mode of declaring victory because “it feels faster,” and it produces the evidence trail required for change control. Verification always compares a fresh measurement against the pre-change baseline.

9.1 Reset and Re-measure the Wait Delta

In a lab or approved window, clear the wait counters, run a representative workload, and re-rank the waits. PAGEIOLATCH should have fallen in both absolute time and rank. In production, prefer a delta approach: snapshot the DMV before and after rather than clearing global counters.

```
/*  LAB / APPROVED WINDOW ONLY -- clears server-wide cumulative wait counters. */
DBCC SQLPERF (N'sys.dm_os_wait_stats', CLEAR);

/*  ... run a representative workload, then re-run the Phase A / 5.2 queries
    and compare PAGEIOLATCH total_wait_ms and avg_wait_ms against baseline. */

/*  PRODUCTION-SAFE delta pattern (no global reset):  */
SELECT wait_type, wait_time_ms, waiting_tasks_count
INTO   #wait_before
FROM   sys.dm_os_wait_stats
WHERE  wait_type LIKE N'PAGEIOLATCH[_]%';

-- (elapse a fixed interval or run the workload)
WAITFOR DELAY '00:05:00';

SELECT  b.wait_type,
        a.wait_time_ms - b.wait_time_ms          AS delta_wait_ms,
        a.waiting_tasks_count - b.waiting_tasks_count AS delta_tasks
FROM    sys.dm_os_wait_stats AS a
JOIN    #wait_before         AS b ON a.wait_type = b.wait_type
ORDER BY delta_wait_ms DESC;
```

9.2 Verification Checklist

Signal	Pre-change	Post-change target
PAGEIOLATCH total wait	Baseline captured in Phase A	Materially reduced or off the top-20
Avg wait per task	Baseline	Reduced, ideally < 10–15 ms

Signal	Pre-change	Post-change target
Logical / physical reads of tuned query	Baseline (STATISTICS IO)	Sharply lower; seek replaces scan
Avg. Disk sec/Read	Baseline PerfMon	Within healthy band for the storage tier
Page Life Expectancy	Baseline	Stable / improved (if memory was the cause)

10. Quick-Reference Decision Tree

Use this condensed logic on-call. Work top to bottom; stop at the first branch that matches and apply the linked phase.

If you observe…	Most likely cause	Go to
High PAGEIOLATCH, LOW avg wait, HIGH task count, plans show scans / missing indexes	Inefficient queries	Phase B (§6)
High PAGEIOLATCH, efficient plans, low/falling PLE, Committed > Target	Memory pressure	Phase C (§7)
High PAGEIOLATCH, HIGH avg wait (> 15–20 ms), efficient plans, adequate memory	Slow storage	Phase D (§8)
Spikes only during backups / index maintenance	Transient bulk I/O	Reschedule; usually benign
PAGELATCH (no IO) in tempdb	Allocation contention (not I/O)	tempdb contention SOP

11. Ongoing Monitoring and Prevention

The cheapest incident is the one that never happens. Fold the following into routine operations so PAGEIOLATCH regressions are caught early rather than during a Sev-1 call.

- Baseline waits, virtual file latency, and PLE on a schedule (e.g. via a dbatools collection job) and retain history to detect drift.
- Alert on sustained average read/write latency crossing the elevated band, not on single spikes.
- Keep statistics and index maintenance current through a scheduled maintenance solution.
- Review the top-I/O plan backlog (§6.2) each maintenance cycle and tune the worst offenders before they escalate.
- Re-validate Max Server Memory after any change to host memory, co-tenancy, or workload profile.
- On cloud IaaS, re-check disk IOPS/throughput provisioning and VM limits whenever the data volume or workload grows materially.

12. Escalation

Escalate beyond the DBA team when the evidence points outside the database engine, and always attach the collected evidence so the receiving team can act immediately.

Confirmed root cause	Escalate to	Attach
Sustained storage latency on healthy plans + memory	Storage / Cloud infrastructure team	Virtual file stats + PerfMon latency + disk tier config
Host-level memory pressure from co-tenants	Server / Virtualization team	sys.dm_os_sys_memory output + host inventory

Confirmed root cause	Escalate to	Attach
Query pattern owned by an application	Application / Development owner	Offending query text + plan + STATISTICS IO

13. Appendix A — Reference Thresholds

Metric	Healthy	Watch	Investigate
Avg read/write latency	< 5 ms	5 – 10 ms	> 10 ms
Avg PAGEIOLATCH wait / task	< 10 ms	10 – 15 ms	> 15–20 ms
Page Life Expectancy	Stable vs. baseline	Gradual decline	Sharp drop vs. baseline
Committed vs. Target memory	Committed ≈ Target	Committed < Target	Committed > Target

Appendix B — Source and Further Reading

This SOP was developed from and expands upon the concepts in “Understanding SQL Server PAGEIOLATCH Wait Types” by M A A Mehedi Hasan (MSSQLTips, 19 Feb 2026):

mssqltips.com/sqlservertip/11611

> **NOTE — Reproduction disclaimer**
> T-SQL diagnostic scripts in this document have been independently rewritten and extended for knowledge sharing and operational use only. They are provided as-is; validate against your environment and run destructive commands only under change control.